

# Laporan Proyek A05 — Tiket Bantuan Pelanggan

Capstone MP-11 · Pola W (work queue) · Anggota: Floribertus Yericho Pramudya, Inkka Ruslly Dwitama, Luqman Adhi Kuncoro · Run bukti utama: `run01` (24 September 2026)

## 1. Masalah, ruang lingkup, dan kriteria keberhasilan

Sistem pada kasus A05 menerima tiket bantuan pelanggan melalui HTTP, meneruskannya sebagai event melalui message broker, lalu menugaskannya ke antrean layanan berdasarkan kategori tiket. Masalah utama yang ingin diselesaikan adalah bagaimana memisahkan producer dan consumer, menjaga pesan ketika worker berhenti, mencegah pemrosesan ganda saat event diulang, dan menolak pesan yang tidak valid.

Cakupan prototipe terdiri dari:

1. Gateway HTTP pada ``gateway/server.js`` untuk menerima ``POST /tickets``.
2. Publikasi event bertipe ``ticket.created`` ke exchange RabbitMQ ``support``.
3. Routing melalui binding ``ticket.created`` menuju queue ``triage``.
4. Worker pada ``layanan/worker.js`` yang memvalidasi, mengklasifikasikan, dan menyimpan tiket.
5. PostgreSQL dengan tabel ``ticket_assignments`` untuk hasil assignment dan ``rejected_messages`` untuk pesan invalid.
6. Skenario uji U1-U4 untuk normal processing, consumer failure, replay/idempotensi, dan validasi pesan.

| Setelah | Assignment unik | FINANCE | TECH | GENERAL | Syarat tambahan                                      |
|---------|-----------------|---------|------|---------|------------------------------------------------------|
| U1      | 20              | 7       | 7    | 6       | himpunan <code>event_id</code> = N01–N20             |
| U2      | 25              | 9       | 9    | 7       | saat worker mati: 5 pesan ready, assignment tetap 20 |
| U3      | 25              | 9       | 9    | 7       | replay N01–N05 tidak menambah baris                  |
| U4      | 26              | 9       | 9    | 8       | X01 tercatat ditolak; V01 selesai                    |

Aturan klasifikasi yang digunakan adalah ``billing`` ke ``FINANCE``, ``technical`` ke ``TECH``, dan kategori lain ke ``GENERAL``.

## 2. Arsitektur dan alasan desain

```
Utama      : Producer → support → [ticket.created] → triage → worker → ticket_assignments
Tidak valid: worker nack(requeue=false) → support.dlx → [rejected] → triage.rejected
```

### 1. Pemisahan gateway dan worker

Gateway hanya menerima request dan menerbitkan event. Proses database berada di worker. Pemisahan ini membuat producer tetap dapat menerima event ketika worker sedang berhenti, sekaligus memungkinkan kapasitas consumer dikembangkan secara terpisah.

### 2. RabbitMQ sebagai buffer durable

Exchange ``support`` menggunakan routing key ``ticket.created`` untuk mengirim event ke queue ``triage``. Exchange dan queue dibuat durable, sedangkan producer mengirim pesan persistent. Dengan

demikian, event yang diterbitkan ketika worker berhenti tetap berada di broker dan dapat diproses setelah worker hidup kembali. Perilaku ini diuji pada U2.

### 3. ACK setelah hasil database berhasil

Worker melakukan `ack` setelah assignment berhasil disimpan dan transaksi PostgreSQL di-commit. Jika terjadi error teknis, pesan di-`nack` dan dikembalikan ke queue. Pola ini mengurangi risiko pesan hilang sebelum hasilnya tersimpan.

### 4. Idempotensi berdasarkan `event\_id`

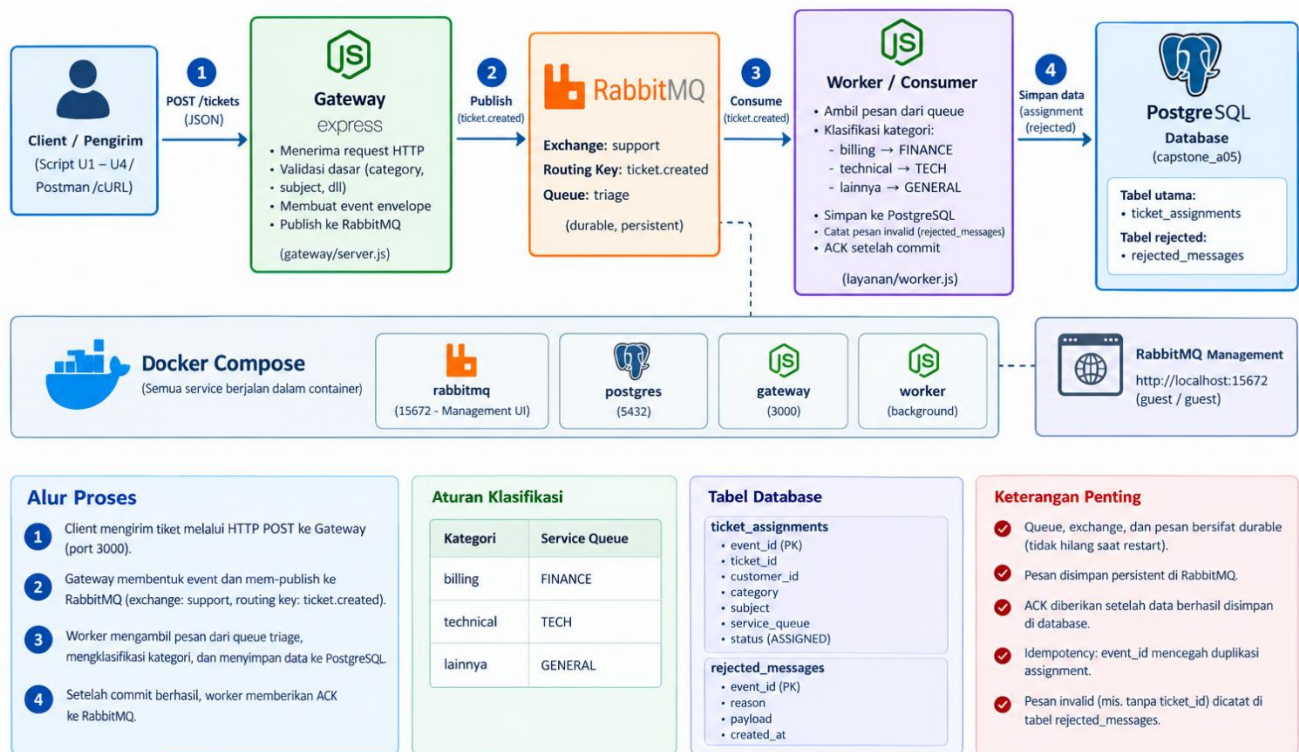
Sebelum insert, worker memeriksa apakah `event\_id` sudah ada di `ticket\_assignments`. Kolom tersebut juga memiliki constraint `UNIQUE`. Jika event sudah pernah diproses, worker melakukan ACK tanpa membuat assignment kedua. Desain ini menjadi dasar pengujian replay pada U3.

### 5. Validasi di gateway dan worker

Gateway menolak request tanpa `ticket\_id` dengan HTTP 400. Worker tetap memiliki validasi sebagai lapisan pertahanan kedua untuk event yang mungkin masuk langsung ke broker. Pesan invalid dicatat pada `rejected\_messages` dan di-ACK agar tidak diproses ulang tanpa batas.

## Diagram Arsitektur – Capstone A05

### Message Broker (RabbitMQ) – Ticketing System



### 3. Hasil pengujian U1–U4

#### U1 - Normal Processing

Script `scripts/u1.js` mengirim 20 event dengan ID `run01-N01` sampai `run01-N20`. Semua event memiliki data tiket yang lengkap. Hasilnya adalah 20 record assignment unik dengan status `ASSIGNED`.

Distribusi kategori menguji ketiga antrean layanan:

- `billing` menghasilkan `FINANCE`.
- `technical` menghasilkan `TECH`.
- `general` menghasilkan `GENERAL`.

**\*\*Hasil U1:\*\*** 20 input, 20 assignment, 0 ID hilang, dan 0 duplikasi.

#### U2 - Consumer Failure dan Recovery

Pada U2, worker dihentikan sebelum lima event `run01-G01` sampai `run01-G05` dikirim. Producer tetap menerima request dengan HTTP 202. Pemeriksaan RabbitMQ menunjukkan lima pesan siap dikonsumsi pada queue `triage`, tanpa pesan unacknowledged dan tanpa consumer aktif.

Setelah worker dijalankan kembali, kelima pesan diproses dan disimpan sebagai assignment. Hasilnya menambah lima record dari kondisi U1, sehingga total menjadi 25 assignment.

**\*\*Hasil U2:\*\*** seluruh ID G01-G05 ditemukan, seluruh status `ASSIGNED`, dan tidak ada pesan yang hilang selama worker berhenti.

#### U3 - Replay dan Idempotensi

Script `scripts/u3.js` mengirim ulang N01-N05 menggunakan `event\_id` dan payload yang sama dengan U1. Semua publish menerima HTTP 202, tetapi worker menemukan bahwa event tersebut sudah ada di database.

Pemeriksaan jumlah record menunjukkan total tetap 25, bukan bertambah menjadi 30. Ini membuktikan bahwa `event\_id` digunakan sebagai identitas pemrosesan dan replay tidak menghasilkan assignment ganda.

**\*\*Hasil U3:\*\*** lima event replay diterima, 0 record assignment baru, dan total tetap 25.

#### U4 - Invalid dan Valid Message

U4 mengirim X01 tanpa `ticket\_id`, kemudian V01 dengan data lengkap. X01 ditolak oleh gateway dengan HTTP 400 dan dicatat sebagai rejected dengan alasan `ticket\_id` wajib diisi. Karena ditolak sebelum publish, X01 tidak masuk ke queue dan tidak membuat retry loop.

V01 diterima dengan HTTP 202, diproses sebagai kategori `billing`, lalu disimpan sebagai `FINANCE / ASSIGNED`. Total assignment akhir menjadi 26, sedangkan `rejected\_messages` berisi satu record untuk X01.

**\*\*Hasil U4:\*\*** X01 rejected, V01 assigned, total akhir 26 assignment dan 1 rejected message.

### ### Rekapitulasi

| Uji | Input utama                  | Hasil                                  | Status |
|-----|------------------------------|----------------------------------------|--------|
| U1  | N01-N20                      | 20 assignment unik                     | LULUS  |
| U2  | G01-G05 saat worker berhenti | Pesan tertahan lalu diproses; total 25 | LULUS  |
| U3  | Replay N01-N05               | Tidak ada duplikasi; total tetap 25    | LULUS  |
| U4  | X01 invalid dan V01 valid    | X01 rejected, V01 assigned; total 26   | LULUS  |

## 4. Diagnosis gangguan dan pemulihan (U2 dan cek03)

Gangguan ditemukan saat U4 ketika event X01 tanpa `ticket\_id` masuk ke worker. Log berulang menunjukkan pola berikut:

```
```text
```

```
INVALID MESSAGE
```

```
Event ID : run01-X01
```

```
Reason : ticket_id wajib diisi
```

```
ERROR processing message: there is no unique or exclusion constraint matching the ON CONFLICT specification
```

```
```
```

### Diagnosis :

1. pesan invalid tetap mencapai worker dan jalur pencatatan rejected gagal karena `ON CONFLICT (event\_id)` belum memiliki constraint yang cocok pada tabel `rejected\_messages`.
2. error tersebut masuk ke blok error umum yang melakukan `nack(..., true)`, sehingga pesan dikembalikan ke queue dan diproses berulang.

Pemulihan dilakukan dengan tiga perubahan:

1. Gateway mengaktifkan validasi `ticket\_id` dan mengembalikan HTTP 400 sebelum event dipublish.
2. Worker menggunakan `ON CONFLICT DO NOTHING` pada pencatatan rejected agar tidak bergantung pada conflict target yang tidak tersedia.
3. Skema `rejected\_messages.event\_id` diberi constraint `UNIQUE` untuk menjaga satu catatan rejection per event.

Setelah perbaikan, X01 dicatat satu kali lalu di-ACK. V01 yang datang sesudahnya tetap diproses normal. Verifikasi database menunjukkan satu rejected message dan 26 assignment. Queue RabbitMQ berakhir dengan `messages\_ready=0` dan `messages\_unacknowledged=0`.

## 5. Batas prototipe

Prototipe ini memiliki beberapa batasan:

1. Belum menyediakan autentikasi dan otorisasi pada endpoint HTTP.
2. Credential default RabbitMQ dan PostgreSQL masih digunakan untuk lingkungan lokal.
3. Belum ada dead-letter exchange, retry policy dengan backoff, atau batas jumlah retry untuk error teknis.
4. Worker berjalan sebagai satu proses dengan `prefetch(1);` belum diuji untuk skala tinggi atau banyak replica.
5. Belum ada observability terpusat seperti metrics, tracing, atau dashboard log.
6. Validasi payload masih minimum, terutama keberadaan `ticket\_id`, `category`, dan `subject`.
7. Migrasi skema masih menggunakan `db/init.sql`, sehingga perubahan pada database volume lama perlu diterapkan secara manual atau dengan migration tool.
8. Tidak ada endpoint untuk query status tiket; pemeriksaan hasil dilakukan langsung melalui PostgreSQL.

Batasan tersebut tidak menghalangi tujuan pengujian A05, tetapi perlu ditangani sebelum sistem dipakai pada lingkungan produksi.

## 6. Kontribusi Anggota

| Anggota                      | Kontribusi                                                                                                             |
|------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Floribertus Yericho Pramudya | Implementasi gateway/producer, endpoint HTTP `/tickets`, pembentukan event, dan publikasi ke RabbitMQ                  |
| Inkka Ruslly Dwitama         | Implementasi worker/consumer, validasi event, klasifikasi antrean, ACK/NACK, serta idempotensi berdasarkan `event_id`. |
| Luqman Adhi Kuncoro          | Implementasi skema PostgreSQL dan Docker Compose, penyusunan skenario U1-U4, dokumentasi, dan evidence.                |
| Bersama                      | Review kode, laporan, dan slide presentasi.                                                                            |

## 7. Lampiran Bukti

Bukti mentah dipisahkan dari laporan utama pada folder [`evidence``](`../evidence`):

- [`evidence/U1``](`../evidence/U1`): normal processing.
- [`evidence/U2``](`../evidence/U2`): consumer failure dan recovery.
- [`evidence/U3``](`../evidence/U3`): replay dan idempotensi.
- [`evidence/U4``](`../evidence/U4`): invalid message dan valid message.

Setiap folder berisi input, output persisten, perbandingan ID, log penting, dan observasi broker. Bukti tersebut dapat digunakan untuk memeriksa klaim hasil tanpa mencampur log mentah ke dalam laporan ringkas.